Project Assignment 3

Compiler Design Lab
Prof. Dr. Sebastian Hack
Fabian Ritter, B.Sc.

Project Task E Semantic Analysis

Implement semantic analysis.

- You can perform this either during parsing and AST construction or as a separate phase.
- Semantic analysis augments `--parse` and `--print-ast`.
- Major parts are name and type analysis. Name analysis associates identifiers with declarations. Type analysis associates expressions with types. It encompasses the Constraints and Semantics clauses.
- If you delayed certain syntactic checks (e.g. rejecting `a || b = c`), perform them now.
- The only *null pointer constant* that you need to support is literal `0`.
- The previous restriction and the restricted language subset ensure that it is not necessary to evaluate the value of any expressions during semantic analysis.
- It is not necessary to accept programs that contain functions which return a struct or have one as parameter. Similarily, it is not necessary to accept programs that contain assignments of struct type.
- It is not necessary to accept programs that contain *anonymous structs* (in the sense this term is used in the standard; struct declarations without tags are still part of our language subset).
- You do not need to handle `__func__`.
- Use `int` for the types `ptrdiff_t` and `size_t`.
- Due to the restricted language subset, type compatibility degenerates to equality.
- For the error location use the location of the (first) terminal of the syntactic construct where the error was detected. E.g. for adding two pointers, show the location of the `+`. For an `if` whose condition is not scalar, show the location of the keyword `if`.
- If you are uncertain about some aspect, ask!
- The soft deadline for this milestone is 2023-01-06.
- Keep it simple!